Data Structures (BCIDS302) Question Bank					
MODULE 1 (Choice of answering)					
Q. No.	**No.**	**Questions**	**Marks**	**BL-CO-PO**	**Topic**
1(a)	1.	Explain the concept of data structures and describe the different types of data structures with suitable examples.	6	L2-CO1-PO1	Topic 1
	2.	Describe the basic data structures operations.	6	L2-CO1-PO1	Topic 1
	3.	Define pointer. With examples, explain pointer declaration.	6	L2-CO1-PO1	Topic 2
	4.	Explain the concept of pointer initialization and use of the pointer in allocating a block of memory dynamically.	6	L2-CO1-PO1	Topic 2
	5.	Explain dangling pointer reference and how to avoid it?	6	L2-CO1-PO1	Topic 2
	6.	Explain the ADT of polynomial.	6	L2-CO1-PO1	Topic 6
	7.	Describe pattern matching and briefly explain the types of pattern matching algorithms.	6	L2-CO1-PO1	Topic 7
	8.	Explain different types of dynamic memory allocation functions with suitable example.	6	L2-CO1-PO1	Topic 2
	9.	Discuss the common errors of dynamic memory allocation.	6	L2-CO1-PO1	Topic 2
	10.	Explain the concept of sparse matrix and illustrate the triplet representation of sparse matrix.	6	L2-CO1-PO1	Topic 5
1(b)	1.	Develop a C program to read ten integers and store them in an array using pointers, print their sum and average.	8	L3-CO2-PO1	Topic 2
	2.	Develop a C program to find the smallest element in an array (using array and function).	8	L3-CO2-PO1	Topic 4
	3.	Develop a C program to exchange the contents of two variables using function (pointers as parameters).	8	L3-CO2-PO1	Topic 2
	4.	Develop a C function to show the working of radix sort with an example.	8	L3-CO2-PO1	Topic 10
	5.	Analyze the logic involved in adding two polynomials and implement a C program to perform the polynomial addition.	8	L3-CO2-PO1	Topic 6
	6.	Develop a program in C to read a sparse matrix of integer values and	8	L3-	Topic

<table>
<tr><td></td><td></td><td>search this matrix for an element specified by the user.</td><td></td><td>CO2-PO1</td><td>5</td></tr>
<tr><td></td><td>7.</td><td>Develop a C program that uses pointers to swap two arrays of equal size without using a temporary array.</td><td>8</td><td>L3-CO2-PO1</td><td>Topic 2</td></tr>
<tr><td></td><td>8.</td><td>Develop a C program to find the pattern in a text using the Naïve Pattern matching algorithm.</td><td>8</td><td>L3-CO2-PO1</td><td>Topic 7</td></tr>
<tr><td></td><td>9.</td><td>Develop a C program using malloc() to read N integers from the user and print the sum of all even numbers.</td><td>8</td><td>L3-CO2-PO1</td><td>Topic 2</td></tr>
<tr><td></td><td>10.</td><td>Develop a C program to dynamically allocate memory for 5 integers using malloc(), then resize the memory to store 10 integers using realloc(), read the values, and display them.</td><td>8</td><td>L3-CO2-PO1</td><td>Topic 2</td></tr>
<tr><td>1(C)</td><td>1.</td><td>Construct the prefix (LPS) table for the pattern string “a b a b.” and analyze how the LPS values influence pattern shifting during mismatches while tracing the execution of the Knuth Morris Pratt (KMP) algorithm on the text “a b a b c a b c a b b b a b d”.</td><td>6</td><td>L4-CO3-PO2</td><td>Topic 8</td></tr>
<tr><td></td><td>2.</td><td>Analyse the code and predict the output
<table>
<tr><td>

```
#include <stdio.h>
int main() {
   int A[3][3] = {
      {0, 5, 0},
      {0, 0, 8},
      {4, 0, 0}
   };
   int B[3][3] = {
      {0, 3, 0},
      {7, 0, 0},
      {0, 6, 0}
   };
   int sum[3][3];

   for(int i=0; i<3; i++) {
      for(int j=0; j<3; j++) {
         sum[i][j] = A[i][j] + B[i][j];
      }
   }
   printf("Sum of Matrices:\n");
   for(int i=0; i<3; i++) {
      for(int j=0; j<3; j++) {
         printf("%d ", sum[i][j]);
      }
      printf("\n");
   }
   return 0;
}
```

</td><td>ii)

```
#include <stdio.h>

int main()
{
   int arr[] = {1, 3, 5, 7, 9};

   int sum = 0;

   for(int i = 0; i < 5; i++)
{
      if(i % 2 == 0)
         sum += arr[i];
      else
         sum -= arr[i];
   }

   printf("%d", sum);
   return 0;
}
```

</td></tr>
</table>
</td><td>6</td><td>L4-CO3-PO2</td><td>Topic 5</td></tr>
</table>

	3.	Analyze the process of arranging the list of elements 8, 3, 2, 7, 9, 1, 4 into ascending order by repeatedly placing each element into its correct position within the already arranged portion of the list. Show the step-by-step working and conclude how many iterations are required to achieve the completely sorted list.	6	L4-CO3-PO2	Topic 9
	4.	Analyse the following declarations: Int a=5; Int b=7; Int *p=&a; Int *q=&b; What is the value of each of the following expressions? i)++a; ii) ++(*p) iii) - -(*q) iv) - -b	6	L4-CO3-PO2	Topic 2
	5.	Analyse the given strings, String= " A B D A B S A B C A B C", Pattern String = "A B C A B C " to generate the prefix table and give the tracing to perform string matching using Knuth Morris Pratt Algorithm(KMP).	6	L4-CO3-PO2	Topic 8
	6.	Analyze the process of arranging the list of elements 4, 3, 2, 10, 12, 1, 5, 6 into ascending order by repeatedly placing each element into its correct position within the already arranged portion of the list. Show the step-by-step working and conclude how many iterations are required to achieve the completely sorted list.	6	L4-CO3-PO2	Topic 9
	7.	Analyse the given strings, String= "C A B A B A B C A B A B A C A", Pattern String = "A B A B A C" to generate the prefix table and give the tracing to perform string matching using Knuth Morris Pratt Algorithm(KMP).	6	L4-CO3-PO2	Topic 8

	8.	Analyse the code and write the final sorted array
		#include <stdio.h>
		int main() {
		int arr[5] = {4, 3, 5, 1, 2};
		for(int i=1; i<5; i++) {
		int key = arr[i];
		int j = i-1;
		while(j>=0 && arr[j] > key) {
		arr[j+1] = arr[j];
		j--;
		}
		arr[j+1] = key;
		}
		for(int k=0; k<5; k++) printf("%d\t", arr[k]);
		return 0;
		}
			6	L4-CO3-PO2	Topic 9

	9.	Analyze the following code and predict its output
		i) #include <stdio.h>
		#include <stdlib.h>
		int main() {
		int *p = (int*)malloc(4 * sizeof(int));
		for(int i=0;i<4;i++)
		*(p+i) = i+1;
		printf("%d\t", *p);
		free(p);
		return 0;
			6	L4-CO3-PO2	Topic 2

		}
		
		ii) #include <stdio.h>
		int main() {
		int arr[] = {1, 2, 3, 4, 5};
		int *p = arr;
		while(*p != 5) {
		printf("%d\t", *p);
		p++;
		}
		return 0;
		}			

	10.	Analyse the code and predict the output
		i) #include <stdio.h>
		int main() {
		int arr[] = {5, 10, 15, 20};
		int *p = arr;
		printf("%d ", *(p+1));
		printf("%d ", *(p+3));
		printf("%d", *p + *(p+2));
		return 0;
		}
		
		ii) #include <stdio.h>
		int main() {
		int arr[] = {4, 8, 12, 16, 20};
		int *p = arr;
		printf("%d ", arr[2]);
		printf("%d ", *(p+4));
		printf("%d ", *p + arr[3]);
		return 0;
		}
			6	L4-CO3-PO2	Topic 2

OR					
2(a)	1.	Describe the concept of data abstraction and illustrate it with a suitable example.	6	L2-CO1-PO1	Topic 3
	2.	Describe Dynamically allocated arrays with suitable example	6	L2-CO1-PO1	Topic 4
	3.	Explain the working of Insertion Sort with suitable example.	6	L2-CO1-PO1	Topic 9
	4.	Describe the working procedure of radix sort with the help of an example	6	L2-CO1-PO1	Topic 10
	5.	Explain string operations with examples	6	L2-CO1-PO1	Topic 11
	6.	Explain the following operations with suitable example (i) String concatenation (ii) String comparison	6	L2-CO1-PO1	Topic 11

	7.	Explain the ADT of an array.	6	L2-CO1-PO1	Topic 4
	8.	Explain ADT of Sparse matrix.	6	L2-CO1-PO1	Topic 4
	9.	Explain ADT of natural numbers.	6	L2-CO1-PO1	Topic 3
	10.	Explain the role of prefix table in making the KMP algorithm efficient.	6	L2-CO1-PO1	Topic 8
2(b)	1.	Develop a C program to check whether the given matrix is sparse matrix or not.	8	L3-CO2-PO1	Topic 5
	2.	Develop a C program to read and print one dimensional array using malloc().	8	L3-CO2-PO1	Topic 4
	3.	Develop a C program to read and print two dimensional array using calloc().	8	L3-CO2-PO1	Topic 4
	4.	Develop a C program for allocation, reallocation and de-allocation of memory.	8	L3-CO2-PO1	Topic 2
	5.	Develop a C program to check whether a given matrix is a sparse matrix and, if so, compute and display its transpose.	8	L3-CO2-PO1	Topic 5
	6.	Develop a C program to sort the integer numbers using insertion sort.	8	L3-CO2-PO1	Topic 9
	7.	Develop a C program to check whether a given string is a palindrome or not , using inbuilt functions	8	L3-CO2-PO1	Topic 11
	8.	Develop a C program to count the frequency of each character in a string using arrays.	8	L3-CO2-PO1	Topic 11
	9.	Develop a C program to insert and delete a number from a given location in an array.	8	L3-CO2-PO1	Topic 4
	10.	Develop a C program to concatenate two strings using pointers.	8	L3-CO2-PO1	Topic 11

2(c)	1.	Analyse the working of the following code snippet:
		int *p, n=5;
		p = (int*)malloc(n * sizeof(int));
		for(int i=0; i<n; i++)
		p[i] = i*i;
		for (int i = 0; i < n; i++)
		printf("p[%d] = %d\n", i, p[i]);
		show the tracing for the same along with the final output.
			6	L4-CO3-PO2	Topic 2

	2.	Analyse the code and predict the output	6	L4-	Topic

		#include <stdio.h>
		#include <string.h>
		int main() {
		char text[] = "ABABABA";
		char pat[] = "ABA";
		int n = strlen(text), m = strlen(pat);
		int count = 0;
		for(int i=0; i <= n-m; i++) {
		int j;
		for(j=0; j<m; j++) {
		if(text[i+j] != pat[j]) break;
		}
		if(j == m) count++;
		}
		printf("%d", count);
		return 0;
		}
				CO3-PO2	7

	3.	Analyze the following code snippet and predict the output
		
		#include <stdio.h>
		int main() {
		int arr[4] = {170, 45, 75, 90};
		int max = arr[0], digits = 0;
		for(int i=1; i<4; i++)
		if(arr[i] > max) max = arr[i];
		while(max > 0) {
		digits++;
		max /= 10;
		}
		printf("%d", digits);
		return 0;
		}
			6	L4-CO3-PO2	Topic 10

	4.	Analyze the given code and predict the output
		i) #include<stdio.h>
		int main()
		{
		int a[] = {1,2,3,4};
		int sum = 0;
		for(int i=0;i<4;i++)
		{
		sum+=a[i];
		}
		printf("%d",sum);
		return 0;
		}
		
		ii) #include<stdio.h>
		void solve()
		{
		int a[] = {1,2,3,4,5};
		int sum=0;
			6	L4-CO3-PO2	Topic 4

		for(int i=0; i<5;i++);
		{
		if(i%2==0)
		sum+=*(a+i);
		else
		sum-=*(a+i);
		}
		printf("%d",sum);
		}
		int main()
		{
		solve();
		return 0;
		}			

	5.	A student applied Radix Sort but only sorted by the units and tens place for the list [100, 12, 45, 201]. Analyze the mistake and explain why the final result is incorrect.	6	L4-CO3-PO2	Topic 10
	6.	Analyse the given string "AABAACAADAABAAABAA", pattern string "AABA" and write the tracing to find the pattern in the string using Naïve Pattern matching algorithm.	6	L4-CO3-PO2	Topic 7
	7.	Analyse the given string "BANANAANANAS", pattern string "ANAN" and write the tracing to find the pattern in the string using Naïve Pattern matching algorithm.	6	L4-CO3-PO2	Topic 2

	8.	Analyze the output after dynamic allocation and initialization
		i) #include <stdio.h>
		#include <stdlib.h>
		int main() {
		int *p = (int*)malloc(3 * sizeof(int));
		for(int i=0; i<3; i++)
		*(p+i) = (i+1) * 2;
		printf("%d %d %d", *p, *(p+1), *(p+2));
		free(p);
		return 0;
		}
		
		ii) #include <stdio.h>
		#include <stdlib.h>
		int main() {
		int n = 5;
		int *arr = (int*)malloc(n * sizeof(int));
		for (int i = 0; i < n; i++)
		arr[i] = (i + 1) * (i + 1);
		for (int i = 0; i < n; i++)
		printf("%d ", arr[i]);
		free(arr);
		return 0;
		}
			6	L4-CO3-PO2	Topic 2

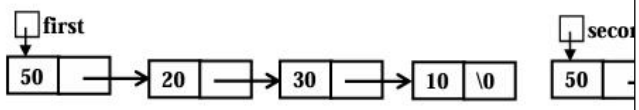
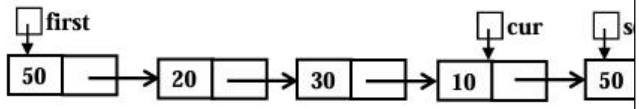
	9.	Analyse the given string "ABABDABACDABABCABAB", pattern string "ABABCABAB" and write the tracing to find the pattern in the string using Naïve Pattern matching algorithm.	6	L4-CO3-PO2	Topic 2
	10.	Analyze in detail the process of arranging the list [170, 45, 75, 90, 802, 24,	6	L4-	Topic

		2, 66] into ascending order using the digit-by-digit mechanism. Break down the step-by-step working for each pass (units, tens, and hundreds place), and show how the elements are distributed and collected at every stage, also finally conclude the number of passes required to obtain the completely sorted list.		CO3-PO2	10
MODULE 2					
3(a)	1	Explain the array representation of a stack with a neat diagram.	5	L2-CO1-PO1	Topic 2
	2	Describe the role of recursion in problem solving with a suitable example.	5	L2-CO1-PO1	Topic 5
	3	Outline the differences between linear queue and circular queue.	5	L2-CO1-PO1	Topic 7, 8
	4	Describe the concept of a priority queue and its applications.	5	L2-CO1-PO1	Topic 10
	5	Explain the operations of a circular queue with an example.	5	L2-CO1-PO1	Topic 8
	6	Outline the advantages and disadvantages of using stack data structure.	5	L2-CO1-PO1	Topic 1
	7	Explain with an example how a queue can be represented using arrays.	5	L2-CO1-PO1	Topic 7
	8	Outline the differences between input-restricted dequeue and output-restricted dequeue.	5	L2-CO1-PO1	Topic 9
	9	Explain how circular queue resolves the limitation of linear queues.	5	L2-CO1-PO1	Topic 8
	10	Explain the various operations performed on stack.	5	L2-CO1-PO1	Topic 1
3(b)	1	Develop a C program to convert infix expression to postfix expression.	8	L3-CO2-PO1	Topic 3
	2	Develop a C program to evaluate the postfix expression.	8	L3-CO2-PO1	Topic 4
	3	Develop a C function to implement the following operations on DeQueue (i) Insert from front (ii) Delete an element from rear	8	L3-CO2-PO1	Topic 9
	4	Develop a C program to show the use of priority queue in CPU job scheduling with at least 4 jobs.	8	L3-CO2-PO1	Topic 10
	5	Develop a C function to implement the following operations on DeQueue (i) Insert from rear (ii) Delete an element from front.	8	L3-CO2-PO1	Topic 9

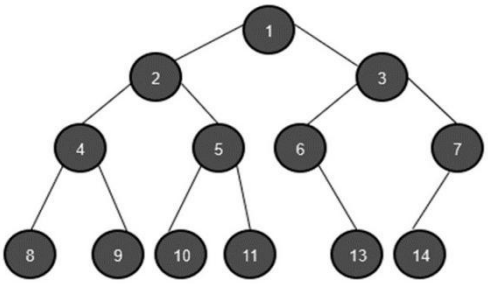
<table>
<tr><td></td><td>6</td><td>Develop a C program to check whether the given string is palindrome or not using stack.</td><td>8</td><td>L3-CO2-PO1</td><td>Topic 2</td></tr>
<tr><td></td><td>7</td><td>Develop a C program to implement Queue operations using arrays.</td><td>8</td><td>L3-CO2-PO1</td><td>Topic 7</td></tr>
<tr><td></td><td>8</td><td>Develop C functions (i) To convert Infix expression into postfix
(ii) Evaluate the postfix expression.</td><td>8</td><td>L3-CO2-PO1</td><td>Topic 4</td></tr>
<tr><td></td><td>9</td><td>Develop a C program to (i) Compute Fibonacci series using recursion (ii) Factorial of a number using recursion.</td><td>8</td><td>L3-CO2-PO1</td><td>Topic 5</td></tr>
<tr><td></td><td>10</td><td>Develop a C program to implement the operations of stack using arrays.</td><td>8</td><td>L3-CO2-PO1</td><td>Topic 2</td></tr>
<tr><td>3(c)</td><td>1</td><td>Analyze the step-by-step stack contents for postfix evaluation of 7 8 + 3 2 + /.</td><td>7</td><td>L4-CO3-PO2</td><td>Topic 4</td></tr>
<tr><td></td><td>2</td><td>Analyze the recursive solution of Tower of Hanoi for 3 disks and show the move sequence.</td><td>7</td><td>L4-CO3-PO2</td><td>Topic 6</td></tr>
<tr><td></td><td>3</td><td>Given the infix expression (A+B)*(C-D), analyse the steps to convert it to postfix and show stack contents.</td><td>7</td><td>L4-CO3-PO2</td><td>Topic 3</td></tr>
<tr><td></td><td>4</td><td>Analyze the step-by-step stack contents for postfix evaluation of 8 2 5 3 + * + 8 - .</td><td>7</td><td>L4-CO3-PO2</td><td>Topic 4</td></tr>
<tr><td></td><td>5</td><td>The contents of a stack S are as follows:
<table><tr><td>Stack (S)</td><td>99</td><td>2</td><td>44</td><td>8</td><td></td><td></td><td></td><td></td></tr><tr><td>Index</td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td></tr></table>The stack can store a maximum of eight elements and the top pointer currently points at index 3. Analyze and show the stack contents by indicating the position of the top pointer after each of the following stack operations:
a) Push(S,5)
b) Push(S,7)
c) Pop(S)
d) Pop(S)
e) Pop(S)
f) Push(S,-1)
g) Push(S,33)</td><td>7</td><td>L4-CO3-PO2</td><td>Topic 2</td></tr>
<tr><td></td><td>6</td><td>The contents of a circular queue Q are as follows:
<table><tr><td>Queue (Q)</td><td>12</td><td>25</td><td>38</td><td></td><td></td></tr></table></td><td>7</td><td>L4-CO3-PO2</td><td>Topic 8</td></tr>
</table>

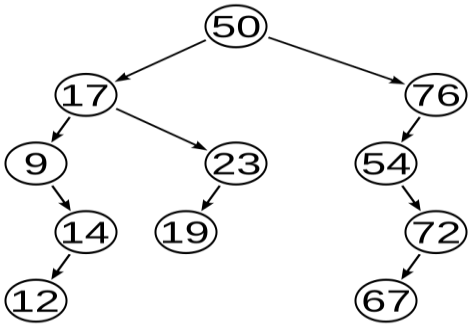
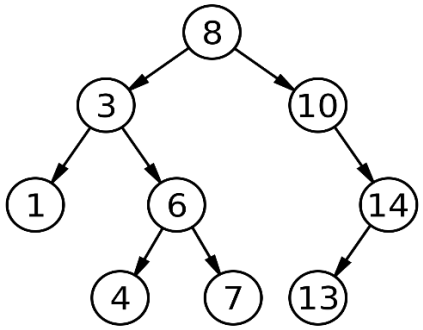
		Index | 0 | 1 | 2 | 3 | 4 The queue can store a maximum of five elements. The Front pointer is currently at index 0, and the Rear pointer is currently at index 2. Perform the following operations in order and, after each step, show the queue diagrammatically, indicating the Front and Rear positions: a) Enqueue(Q,50) b) Enqueue(Q,65) c) Dequeue(Q) d) Enqueue(Q,75) e) Dequeue(Q) f) Dequeue(Q) g) Enqueue(Q, 90)			
	7	Consider the following two states of a stack S State 1 *Stack(S)* | *99* | *2* | *44* | *-1* | | | | | *Index* | *0* | *1* | *2* | *3* | *4* | *5* | *6* | *7* *State 2* *Stack(S)* | *99* | *2* | *-1* | *6* | 7 | 31 | | | *Index* | *0* | *1* | *2* | *3* | *4* | *5* | *6* | *7* Analyse and write the series of push and pop operations that will transition the stack S from State 1 to State 2.	7	L4-CO3-PO2	Topic 2
	8	A dequeue of size 5 currently has [15, 20, 25] with Front=0, Rear=2. Analyze and show the contents of the deque after performing the following operations. After every operation, indicate the position of Front and Rear pointers. InsertFront(10), InsertRear(30), DeleteFront(), DeleteRear(), Insertfront(22), InsertRear(45), DeleteRear().	7	L4-CO3-PO2	Topic 9
	9	Given the infix expression A+B*(C-D)/E, analyse the steps to convert it to postfix and show stack contents.	7	L4-CO3-PO2	Topic 3
	10	The contents of a circular queue Q are as follows: Queue (Q) | 33 | 12 | 3 | 45 | Index | 0 | 1 | 2 | 3 | 4 The queue can store a maximum of five elements. The Front pointer is currently at index 0, and the Rear pointer is currently at index 3. Perform the following operations in order and, after each step, show the queue	7	L4-CO3-PO2	Topic 8

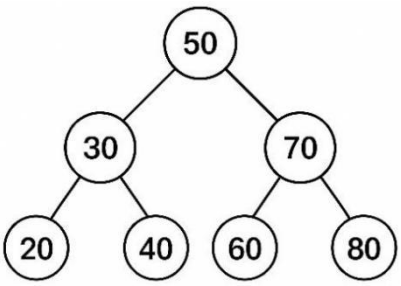
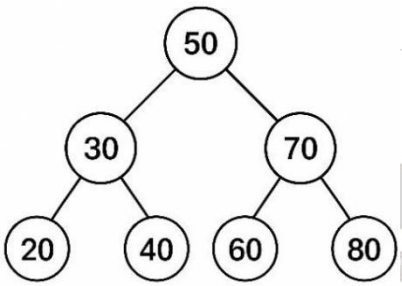
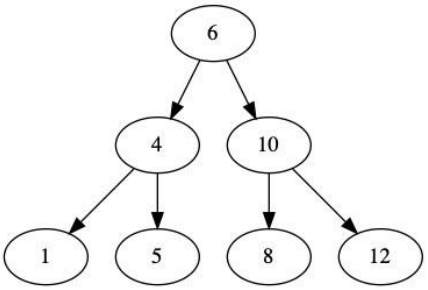
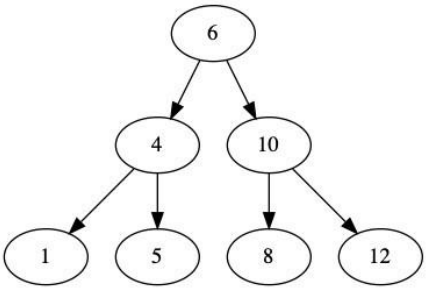
		diagrammatically, indicating the Front and Rear positions: a) Enqueue(Q,33) b) Dequeue(Q) c) Enqueue(Q,25) d) Dequeue(Q) e) Dequeue(Q) f) Enqueue(Q, 98) g) Enqueue(Q,66)			
MODULE 3					
4(a)	1	Explain the representation of a singly linked list in memory with a neat diagram.	6	L2-CO1-PO1	Topic 2
	2	Outline the differences between arrays and linked lists.	6	L2-CO1-PO1	Topic 1
	3	Explain the concept of header linked list with suitable example.	6	L2-CO1-PO1	Topic 6
	4	Explain the advantages of using linked lists over arrays.	6	L2-CO1-PO1	Topic 1
	5	Outline the differences between singly linked list and doubly linked list.	6	L2-CO1-PO1	Topic 7
	6	Explain the concept of circular linked list with suitable example.	6	L2-CO1-PO1	Topic 10
	7	Explain advantages and disadvantages of a circular linked list.	6	L2-CO1-PO1	Topic 10
	8	Outline the differences between header linked list and ordinary linked list.	6	L2-CO1-PO1	Topic 6
	9	Outline the differences between header linked list and circular linked list.	6	L2-CO1-PO1	Topic 6/10
	10	Explain the following with suitable diagrams: (i) Singly linked list (ii) Doubly Linked list (iii) Circular linked list.	6	L2-CO1-PO1	Topic 1/7/10
4(b)	1	Develop a C program to create a singly linked list, insert 5 elements and display its elements.	8	L3-CO2-PO1	Topic 4
	2	Develop a C functions to perform the following operations on circular linked list: (i) Insertion into circular linked list (ii) Deletion of an element from the circular linked list.	8	L3-CO2-PO1	Topic 10
	3	Develop a C program to insert a node at a specified position in a singly	8	L3-	Topic

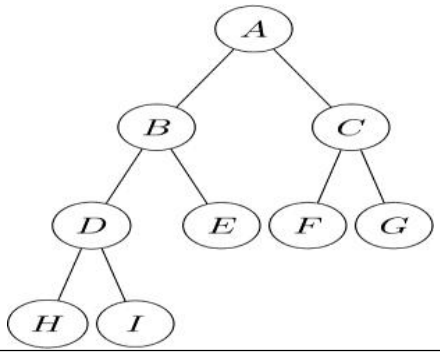
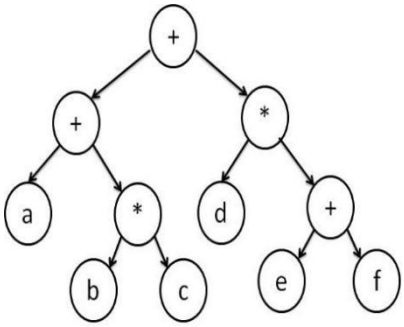
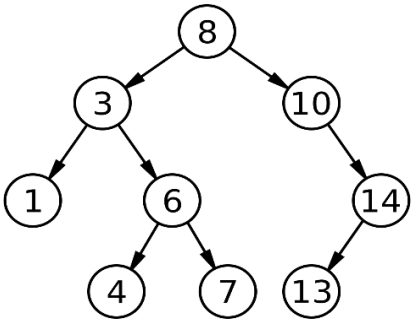
		linked list.		CO2-PO1	4
	4	Develop C functions to implement the following operations in doubly linked list: (i) Deletion from front end (ii) insertion at rear end.	8	L3-CO2-PO1	Topic 4/5
	5	Develop a C program to implement stack operations using a linked list.	8	L3-CO2-PO1	Topic 8
	6	Develop a C program to delete a node from a doubly linked list given a key.	8	L3-CO2-PO1	Topic 5
	7	Develop C functions to implement the following operations in singly linked list: (i) Deletion from rear end (ii) insertion at rear end.	8	L3-CO2-PO1	Topic 4 / 5
	8	Develop a C program to implement queue operations using singly linked list.	8	L3-CO2-PO1	Topic 9
	9	Develop a C program to insert a node before a given node in a doubly linked list.	8	L3-CO2-PO1	Topic 4
	10	Develop a C functions to perform the following operations on singly linked lists: (i) Intersection of two lists (ii) Union of two lists.	8	L3-CO2-PO1	Topic 11/12
4(c)	1	Analyse the given input scenario and implement appropriate C function to get the desired output. **Input Lists:**  **Output List:** 	6	L4-CO3-PO2	Topic 4/5
	2	Analyze the below scenario and implement appropriate C function for the same: You are given the pointers to the head nodes of two linked lists. Compare the data in the nodes of the linked lists to check if they are equal. If all data attributes are equal and the lists are the same length, return 1. Otherwise, return 0.	6	L4-CO3-PO2	Topic 3
	3	Analyze the characteristics of a sorted singly linked list to determine how duplicate elements can be efficiently identified and removed. Considering all possible cases (empty list, single-node list, multiple consecutive duplicates), explain the logic required to eliminate duplicate nodes while preserving the sorted order. Based on your analysis, implement a C function that deletes duplicate nodes and returns a list containing only distinct values.	6	L4-CO3-PO2	Topic 5
	4	Analyse the given input scenario and implement appropriate C function to get the desired output.	6	L4-CO3-	Topic 4

		Input: List 1: 1 → 3 → 5 → 7 → NULL Output: 1 → 2 → 3 → 2 → 5 → 2 → 7 → 2 → NULL		PO2	
	5	Analyze the given input scenario and implement a C function for the same. Input: 10 → 20 → 30 → 40 → 50 → NULL Value: 30 Expected Output: Value 30 found at position 3	6	L4-CO3-PO2	Topic 3
	6	Analyze the given scenario and implement a C function for the same: Input: [10] → [20] → [30] → [40] → NULL Expected Output: [40] → [30] → [20] → [10] → NULL	6	L4-CO3-PO2	Topic 4
	7	Analyze the given scenario and implement a C function for the same: Input: [10] ⇄ [20] ⇄ [30] → NULL Expected Output: [5] ⇄ [10] ⇄ [20] ⇄ [30] → NULL	6	L4-CO3-PO2	Topic 7
	8	Analyze the given input scenario with respect to doubly linked list and implement a C function for the same. Input: [10] ⇄ [20] ⇄ [30] → NULL Output: [10] ⇄ [20] → NULL	6	L4-CO3-PO2	Topic 5,7
	9	Analyse the given input scenario and implement appropriate C function to get the desired output. Input: List 1: 1 → 3 → 5 → 7 → NULL Output: 1 → 2 → 3 →4 → 5 → 6 → 7 → 8 → NULL	6	L4-CO3-PO2	Topic 4
	10	Analyse the given input scenario and implement appropriate C function to get the desired output. Input: 1 → 1 → 2 → 3 → 3 → 4 → 4 → 5 → NULL Output: 1 → 2 → 3 → 4 → 5 → NULL	6	L4-CO3-PO2	Topic 5
Module 4					
5(a)	1	Explain the concept of threaded binary tree in detail.	5	L2-CO1-PO1	Topic 5
	2	Explain the properties of binary trees.	5	L2-CO1-PO1	Topic 2
	3	Explain strictly binary tree and complete binary tree.	5	L2-CO1-PO1	Topic 4
	4	Explain the different types of binary trees with suitable example.	5	L2-CO1-PO1	Topic 2
	5	Define the following terms: (i) Root (ii) Leaf nodes (iii) Height of a tree (iv) Siblings	5	L2-CO1-PO1	Topic 1

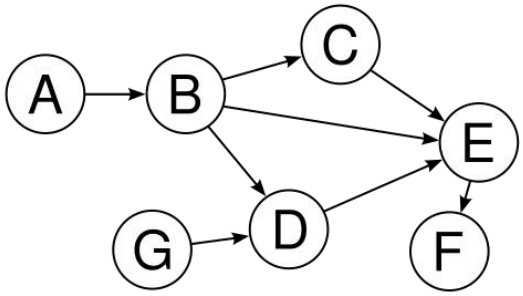
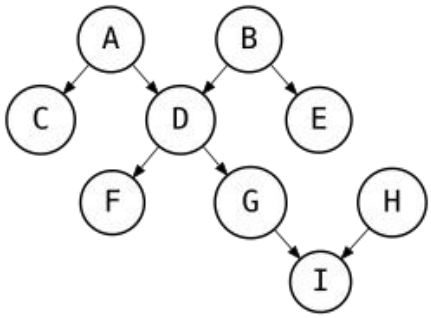
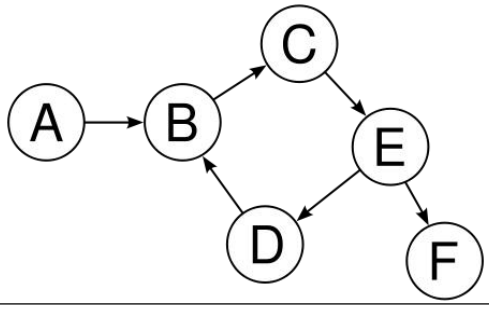
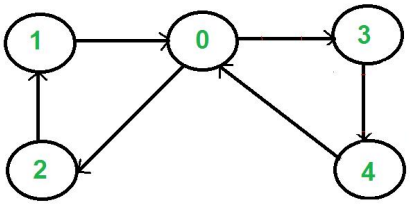
		(v) Subtree			
	6	Explain any five basic terminologies used in binary tree with suitable example.	5	L2-CO1-PO1	Topic 2
	7	Explain the advantage of threaded binary tree over normal binary tree with suitable example.	5	L2-CO1-PO1	Topic 5
	8	Explain how a binary tree is represented using doubly linked list with suitable example.	5	L2-CO1-PO1	Topic 2
	9	Differentiate complete binary tree and almost complete binary tree with suitable example.	5	L2-CO1-PO1	Topic 4
	10	Explain the concept of expression tree with suitable example.	5	L2-CO1-PO1	Topic 7
5(b)	1	Construct a Binary search tree for the given data: 100,85,45,55,110,20,70,65 and perform tree traversal techniques for the same.	10	L3-CO2-PO1	Topic 6
	2	Apply the following traversal operations on given tree: (i) In-order (ii) Pre-order (iii) Post-order 	10	L3-CO2-PO1	Topic 3
	3	Construct an expression tree for the expression ((a + ((b/c)*d))- e) and perform tree traversal techniques for the same.	10	L3-CO2-PO1	Topic 7
	4	Design a C function for the following tree traversals techniques: (i) In-order (ii) Pre-order (iii) Post-order	10	L3-CO2-PO1	Topic 3
	5	Construct a binary search tree for the given set of values: 14,15,4,9,7,18,3,5,16,20 and write a C function to insert an element in BST.	10	L3-CO2-PO1	Topic 6

	6	Develop a C program to construct binary search tree and perform the traversal of the same.	10	L3-CO2-PO1	Topic 6
	7	Construct a binary search tree for the given set of values: 3, 5, 2, 1, 4, 6 and convert the same tree to threaded binary tree to avoid memory wastage.	10	L3-CO2-PO1	Topic 5
	8	Apply the following BST operations on given tree: (i) Height of Tree (ii) In-order (iii) Pre-order (iv) Post-order 	10	L3-CO2-PO1	Topic 6
	9	Construct an expression tree for the expression 3+4*(7-6)/4+3 and perform tree traversal techniques for the same.	10	L3-CO2-PO1	Topic 7
	10	Convert the given binary search tree to threaded binary tree and perform tree traversal techniques for the same. 	10	L3-CO2-PO1	Topic 6
5(c)	1	Analyse the following in-order and post-order traversal and construct pre-order traversal Sequence. In-order: 4, 8, 2, 5, 1, 6, 3, 7 Post-order: 8, 4, 5, 2, 6, 7, 3, 1	5	L4-CO3-PO2	Topic 3
	2	Analyse the following in-order and pre-order traversal and construct post order traversal Sequence. In-order: D, B, E, A, F, C Pre-order: A, B, D, E, C, F	5	L4-CO3-PO2	Topic 3

	3	Analyse the given binary search tree and develop a C function to find the maximum node value present in it. 	5	L4-CO3-PO2	Topic 6
	4	Analyse the given binary search tree and develop a C function to find the minimum node value present in it. 	5	L4-CO3-PO2	Topic 6
	5	Analyse the given binary search tree and develop a C function to count the number of leaf nodes present in it. 	5	L4-CO3-PO2	Topic 6
	6	Analyse the given binary search tree and develop a C function to count the number of nodes(including root node) present in it. 	5	L4-CO3-PO2	Topic 6

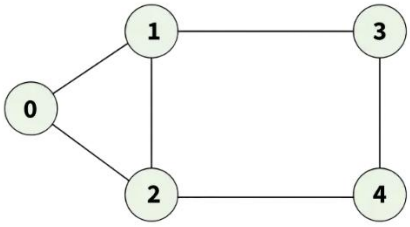
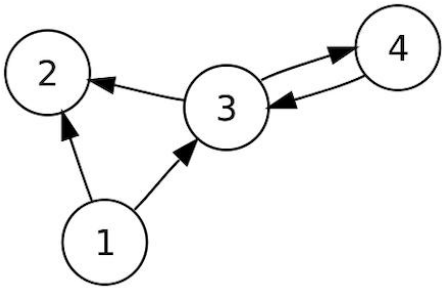
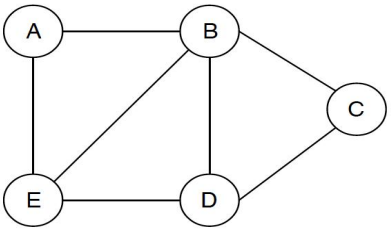
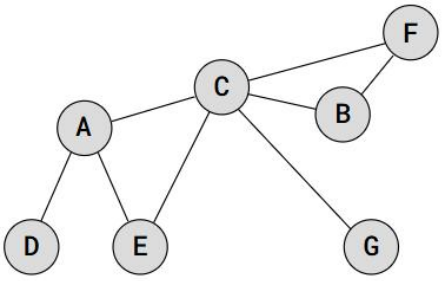
	7	Analyse the given binary search tree and develop a recursive function to find the height of the tree. 	5	L4-CO3-PO2	Topic 6
	8	Analyse the given in-order and post-order traversal and construct pre order traversal Sequence. In-order: B, D, A, E, F, C Post-order: D, B, F, E, C, A	5	L4-CO3-PO2	Topic 3
	9	Analyse the given expression tree and generate the infix, postfix, prefix expression from the same. 	5	L4-CO3-PO2	Topic 7
	10	Consider the following binary search tree:  Design a C function to find a given key in a Binary Search Tree. If the key is found, print "Key Found", otherwise print "Key Not Found".	5	L4-CO3-PO2	Topic 6
Module 5					
6 (a)	1.	Explain the following with suitable examples for each i) Types of Graphs ii) Representation of Graphs	6	L2-CO1-PO1	Topic 6

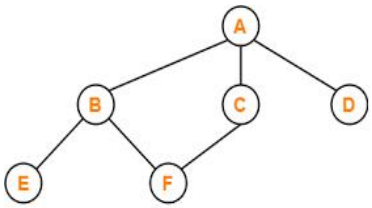
	2.	Explain the following with a suitable example i) Mid Square Method ii) Counting keys to integers	6	L2-CO1-PO1	Topic 8
	3.	Explain the following hashing techniques with a suitable example: i) Division Method ii) Folding Method	6	L2-CO1-PO1	Topic 8
	4.	Explain how to represent graph storage using Adjacency matrix with suitable example.	6	L2-CO1-PO1	Topic 6
	5.	Explain linear probing and chaining hashing techniques to avoid collisions, with examples.	6	L2-CO1-PO1	Topic 8
	6.	Explain directed and undirected graph with suitable example.	6	L2-CO1-PO1	Topic 6
	7.	Explain the concept of Collision with suitable example.	6	L2-CO1-PO1	Topic 8
	8.	Explain the following i)Hashing ii)Hash table	6	L2-CO1-PO1	Topic 8
	9.	Explain the different types of hashing functions.	6	L2-CO1-PO1	Topic 8
	10.	Explain how BFS is different from DFS with a suitable example and justification.	6	L2-CO1-PO1	Topic 6
6.b	1.	Develop 'C' function to implement the following (i) create a node (ii) Insert a new node into AVL tree.	8	L3-CO2-PO1	Topic 1
	2.	Develop 'C' function to perform (i) Left rotation (ii) Right rotation in AVL trees.	8	L3-CO2-PO1	Topic 1
	3.	Develop a 'C' function to handle collision in static hashing using linear probing technique.	8	L3-CO2-PO1	Topic 8
	4.	Develop a 'C' program to print the reachable nodes of a graph from the source node using DFS method.	8	L3-CO2-PO1	Topic 6
	5.	Develop a 'C' function to Search a key element in Patricia.	8	L3-CO2-PO1	Topic 4
	6.	Apply DFS Algorithm to the following graph to display the nodes reachable from source vertex A.	8	L3-CO2-PO1	Topic 6

					
	7.	Apply DFS Algorithm to the following graph to display the nodes reachable from source vertex B. 	8	L3-CO2-PO1	Topic 6
	8.	Apply DFS Algorithm to the following graph to display the nodes reachable from source vertex A. 	8	L3-CO2-PO1	Topic 6
	9.	Develop a 'C' function to finding an Optimal Binary Search tree.	8	L3-CO2-PO1	Topic 10
	10.	Apply DFS Algorithm to the following graph to display the nodes reachable from source vertex 0. 	8	L3-CO2-PO1	Topic 6
6.c	1.	Analyze the rotation type required after inserting a node 25 into the AVL Tree. Explain your reasoning step-by-step. 30 / 20 / 10	6	L4-CO3-PO2	Topic 1

	2.	After inserting the nodes 100 and 200, analyze how rebalancing affects multiple levels of the AVL Tree. Is any rotation required? Justify your analysis with updated balance factors and structure. 50 / \ 30 70 / \ / \ 20 40 60 80	6	L4-CO3-PO2	Topic 1
	3.	Construct AVL Tree 10, 20, 30,40,50 for the given sequence and analyze how the balance restoration differs in each node insertion. Discuss the rotation(s) used during the tree construction.	6	L4-CO3-PO2	Topic 1
	4.	Given AVL Tree: 20 / 10 Identify the imbalance after the insertion of node 15 and analyze why a single rotation won't suffice. Trace the double rotation (Left-Right) required to balance the tree and show the final balanced tree.	6	L4-CO3-PO2	Topic 1
	5.	Insert the values in sequence: 50, 30, 70, 10, 40, 60, 80, 5. After inserting all values, analyse the height of the AVL Tree and identify where (if any) rebalancing occurred. Explain how AVL's height-balancing property is preserved throughout.	6	L4-CO3-PO2	Topic 1
	6.	Given the sequence of nodes 10,20,30,15,25,5 analyze step-by-step how the Red-Black Tree is constructed. Identify each rotation and color change during insertion and explain why they are necessary.	6	L4-CO3-PO2	Topic 2
	7.	If the nodes 40,10,20,30,50 are inserted into an empty Red-Black Tree, analyze the final tree structure and explain how the Red-Black properties guide the rebalancing after each insertion.	6	L4-CO3-PO2	Topic 2
	8.	Given a set of nodes 7,3,18,10,22,8,11,26, analyse how the Red-Black Tree properties are maintained after each insertion. Identify which insertion causes violations and describe how these violations are resolved using rotations and color flips.	6	L4-CO3-PO2	Topic 2
	9.	Given the set of binary strings 1010,1001,1011,1100, analyze the step-by-step construction of a Binary Trie. Explain how the structure evolves with each insertion.	6	L4-CO3-PO2	Topic 2
	10.	For the set of keys 0011,0110,0100,0111 analyze how the Patricia Trie is constructed. Identify how path compression reduces the number of nodes compared to a standard Binary Trie and explain the effect this has on search efficiency.	6	L4-CO3-PO2	Topic 4
OR					
7.a	1.	Explain the operations related to Patricia.	6	L2-CO1-PO1	Topic 4
	2.	Explain the various ADT of Graph.	6	L2-CO1-PO1	Topic 5
	3.	Explain the Optimal Binary Search Tree with a suitable example.	6	L2-CO1-PO1	Topic 10

	4.	Explain the differences between static and dynamic hashing.	6	L2-CO1-PO1	Topic 8 & 9
	5.	Explain the different properties of Red Black Tree.	6	L2-CO1-PO1	Topic 2
	6.	Explain Digital Search Tree with suitable example.	6	L2-CO1-PO1	Topic 3
	7.	Explain the working of a trie and its applications in information retrieval.	6	L2-CO1-PO1	Topic 4
	8.	Explain the different types of rotations in AVL trees with diagrams.	6	L2-CO1-PO1	Topic 1
	9.	Explain Compressed Binary Tries with suitable example.	6	L2-CO1-PO1	Topic 4
	10.	Explain Binary tries with suitable example.	6	L2-CO1-PO1	Topic 4
7.b.	1.	Construct an AVL tree for the given sequence 45,70,35,3,74,25,81,60.	8	L3-CO2-PO1	Topic 1
	2.	Develop a 'C' program to print the reachable nodes of a graph from the source node using BFS method.	8	L3-CO2-PO1	Topic 6
	3.	Identify the various types of imbalances that occur during the construction of AVL Tree with an example.	8	L3-CO2-PO1	Topic 1
	4.	Develop a 'C' function to insert a key into a Patricia.	8	L3-CO2-PO1	Topic 4
	5.	Develop a 'C' function to print the height and balance factor of an AVL tree.	8	L3-CO2-PO1	Topic 1
	6.	Develop the Breadth-First Search (BFS) algorithm for traversing a graph with an example	8	L3-CO2-PO1	Topic 6
	7.	Construct an AVL tree for the given elements 21,26,30,9,4,14,28.	8	L3-CO2-PO1	Topic 6
	8.	Develop the Depth-First Search (DFS) algorithm for traversing a graph with an example.	8	L3-CO2-PO1	Topic 1

	9.	Construct an AVL tree for the given sequence 10, 20, 30, 40, 50, 25.	8	L3-CO2-PO1	Topic 1
	10.	Develop a 'C' functions to handle various types of imbalances in AVL Trees.	8	L3-CO2-PO1	Topic 1
7.c.	1.	Analyse the given graph using BFS and display the nodes reachable from source vertex 0. 	6	L4-CO3-PO2	Topic 6
	2.	Analyse the given graph using BFS and display the nodes reachable from source vertex 1. 	6	L4-CO3-PO2	Topic 6
	3.	Analyse the given graph using BFS and display the nodes reachable from source vertex C. 	6	L4-CO3-PO2	Topic 6
	4.	Analyse the given graph using BFS and display the nodes reachable from source vertex A. 	6	L4-CO3-PO2	Topic 6

	5.	Analyse the given graph using BFS and display the nodes reachable from source vertex A.  	6	L4-CO3-PO2	Topic 6
	6.	Given a set of elements 10, 20, 30, 25, 15, 5, analyze how division method and mid square method hash functions affect the distribution of keys in the hash table.	6	L4-CO3-PO2	Topic 8
	7.	Given a set of integer keys 15,23,7,9,31,14 and a hash table of size 10, analyse the effectiveness of open addressing collision resolution techniques i.e., linear probing.	6	L4-CO3-PO2	Topic 8
	8.	Analyze how linear probing technique handles collisions when inserting a given dataset 12,22,32,42,52,62 into a hash table of size 10 using division method.	6	L4-CO3-PO2	Topic 8
	9.	Analyse how linear probing resolve collisions when inserting the dataset 14,24,34,44,54,64 into a hash table of size 10 using folding method.	6	L4-CO3-PO2	Topic 8
	10.	Analyse the process of constructing an Optimal Binary Search Tree by considering any 5-integer data.	6	L4-CO3-PO2	Topic 10